UNIVERSITY OF COLORADO - BOULDER
Robotics Program

ROBO 5000 (CSCI 5202) - Intro to Robotics

Challenge Problem #2 (Assigned: Friday 09/25, Due: Monday 10/19 11:59pm on *Canvas*)
Non-Holonomic Path Planning

Instructions

Create a new private repository on one group member's GitHub accounts that is to be shared with the other group members, as well as the course instructor and graders (calebescobedo, brsrikrishna). The repository should be named "CP2_GroupName", where GroupName is a group name that you are free to choose. You will ultimately submit a link to this repository as your final submission for the Challenge Problem (groups are formed on Canvas; each group submits a single link). This repository should be well organized and include all of your code, instructions on how to run it, and your final report PDF.

For all challenge problems, your group is expected to turn in a **report** that is 1-4 pages in length. The exact content for this report is not prescribed, but should include details on your overall approach to the challenge problem, the overall results of your solution and whether it was successful or not, any problems encountered and how you overcame them (or approached that you took if you weren't able to overcome them), relevant figures that illustrate the content of your report or show your results. Any references or citations you used for your solution or report should also be included at the end of the report.

1 Pybullet

PyBullet is a Python module for physics simulation, robotics, and deep reinforcement learning, built on the Bullet Physics SDK. It allows us to simulate rigid body dynamics, collision detection, and robot control in virtual environments. You will be using pybullet in the next couple Challenge Problems.

You can setup pybullet by either installing the library directly, or setting up a virtual conda environment. If you are running pybullet on a Mac, you may run into issues using pip install and will have to install a conda environment.

1.1 Installing Pybullet with pip

`https://pypi.org/project/pybullet/`

1.2 Installing Pybullet with Conda

1. Downlood and setup Conda: `https://www.anaconda.com/download/`
2. Create a new conda environment:

   ```
   conda create --name pybulletenv
   ```

3. Activate your conda environment:

   ```
   conda activate pybulletenv
   ```

Note: Everytime you open a new terminal, you will have to reactivate your conda environment to run your script.

4. Install Pybullet: `https://anaconda.org/conda-forge/pybullet`

```
conda install conda-forge::pybullet
```

2 Nonholonomic RRT

2.1 Planning - 100 points

You have been provided with starter code to setup the pybullet environment that consists of a turtlebot in an enclosed maze. The goal state for the robot is (x,y,z) = (4.5, 4.5, 0).

Your goal is to implement RRT with 50% goal bias to plan a path from the robot's initial state to the goal region. For this purpose, the randomly sampled state is sampled within the arena dimensions. Once sampled, use 5 collision-free Monte-Carlo propagations from the robot's nearest node in your tree with varying actions (linear and angular velocity inputs) and time durations. Pick the nearest propagation (measured by Euclidean distance) that leads the robot the closest to the randomly sampled state and add that to the tree (discard all other propagations). Additionally, discard propagation branches that result in the robot being in collision with an obstacle.

Once the tree reaches the goal region, which would be a circle of radius 0.3 from the goal state, stop the simulation and path finding algorithm. In addition to the solution code, attach a video or a GIF of the robot performing the RRT search.

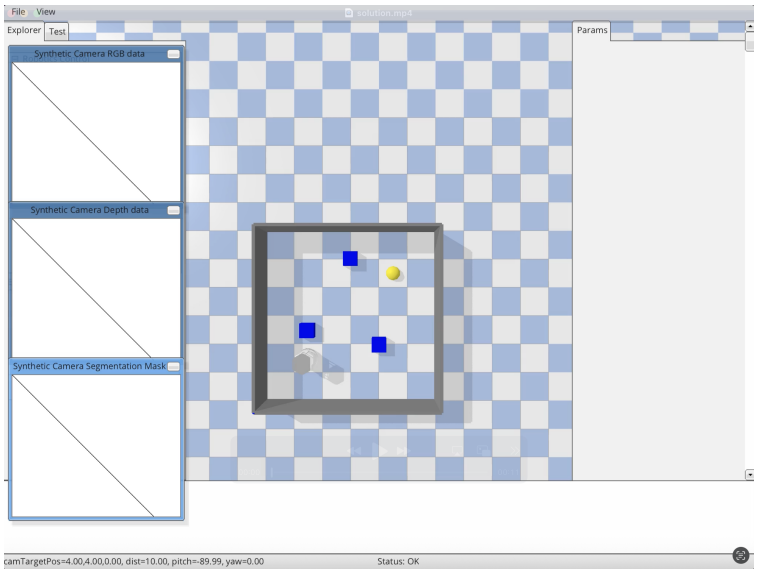


Figure 1: Example of pybullet world. The blue cubes represent obstacles to navigate around. The yellow sphere denotes the goal state (it is just a visual marker so you will not collide into the sphere).

2.2 Deploying the solution - 5 bonus points

Once the tree reaches the goal region, which would be a circle of radius 0.3 from the goal state, reset the simulation and deploy the series of propagations that lead the turtlebot from its initial state to the goal state. In addition to the solution code, attach a video or a GIF of the robot deploying your solution path.

2.3 Helpful Plotting Functions

There are two functions that have been included in the Turtlebot Class. The first is plot_path. This function takes in a list of x, y, z coordinates that represent the points on a potential trajectory. The second is show_intermediate_goal. This function takes in a single x, y, z coordinate and plots a blue cross mark at that location. This can help you see where you are planning when you sample a random point. Remember, your algorithm only generates a random point if it isn't sampling the final goal position. Using these two functions, please produce a plot similar to the one below in 2. Here the green path is the bonus points path. However, just plotting the green path isn't sufficient for the bonus. You must play back the path and make your Turtlebot follow it.

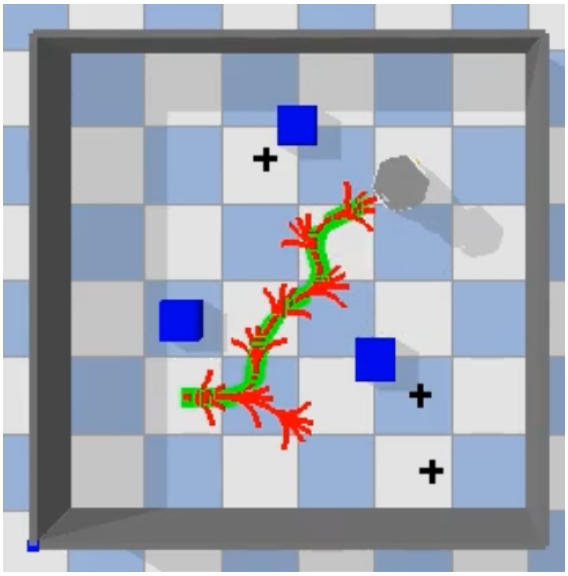


Figure 2: An example of an expected plotted solution. Black crosses are old explored goals. Red trajectories are potential and explored paths. Green path was final solution.